



Graphic Era
deemed to be **University**
DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

Satellite Health Monitoring and Collision Avoidance

Team Name: Team #	Sentra DSCPP-III-2026-T004
Team member 1 (Team Lead)	Chhavit – 2510011425 2510011425@geu.ac.in 
Team member 2	Jain, Harsh – 2510370400 2510370400@geu.ac.in 
Team member 3	Kumar, Satyam – 2510380092 2510380092@geu.ac.in 
Team member 4	Ojha, Sudhanshu – 2510011648 2510011648@geu.ac.in 

PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

Our project, Satellite Health Monitoring and Collision Avoidance, is a C++ simulation that tackles the growing issue concerning space junk in Earth's orbit. The satellites have to avoid the debris and at the same time automatically correct any internal errors, not waiting for commands from humans.

We built the backend of this system by combining Data Structures (DSA) and Object-Oriented Programming concepts from the syllabus of the third semester. The satellites keep track of their own battery life and fuel, correct the software bugs, and achieve circular orbits using simulated gravity. The system makes use of spatial tracking (AVL). We also included a function for linear search in order to locate certain satellites at the end of the simulation.

In Phase III our plan is to switch from a terminal-based system. In order for this to be demonstrated the simulation should be shown using a visual HTML/CSS frontend web application since this makes it much easier to demonstrate.

Updated Project Approach and Architecture

We designed the project using a modular C++ structure, dividing the code into multiple header files based on our 3rd-semester syllabus topics. This allowed us to work in parallel without causing Git merge conflicts.

1. Data Structures (DSA):

Satyam developed AVL_Tree.h to divide space into 3D space, allowing the program to quickly search for debris only in nearby area. Sudhanshu built Queues_DSA.h, which uses a Min-Heap to prioritize the most dangerous debris and a Circular Queue to log battery history.

2. Object-Oriented Logic (OOP):

Chhavit developed Satellites_OOP.h, using base and derived classes to represent different satellites, active fuel consumption. We also created FlightComputer.h to handle the decision-making logic, calculating collision risk and triggering evasive burns.

3. Simulation Mechanics & Input Validation:

Harsh developed Chaos_Engine.h, which randomly generates hardware faults and new debris. We also implemented strict input buffer to prevent the program from crashing if users type invalid data, and added a linear search tool to find final satellite coordinates.

4. Future Phase III Architecture:

We updated the simulation to output a telemetry.csv file. This csv logs the exact X, Y, Z coordinates of every object, acting as a bridge to input data into our future HTML/CSS visual frontend.

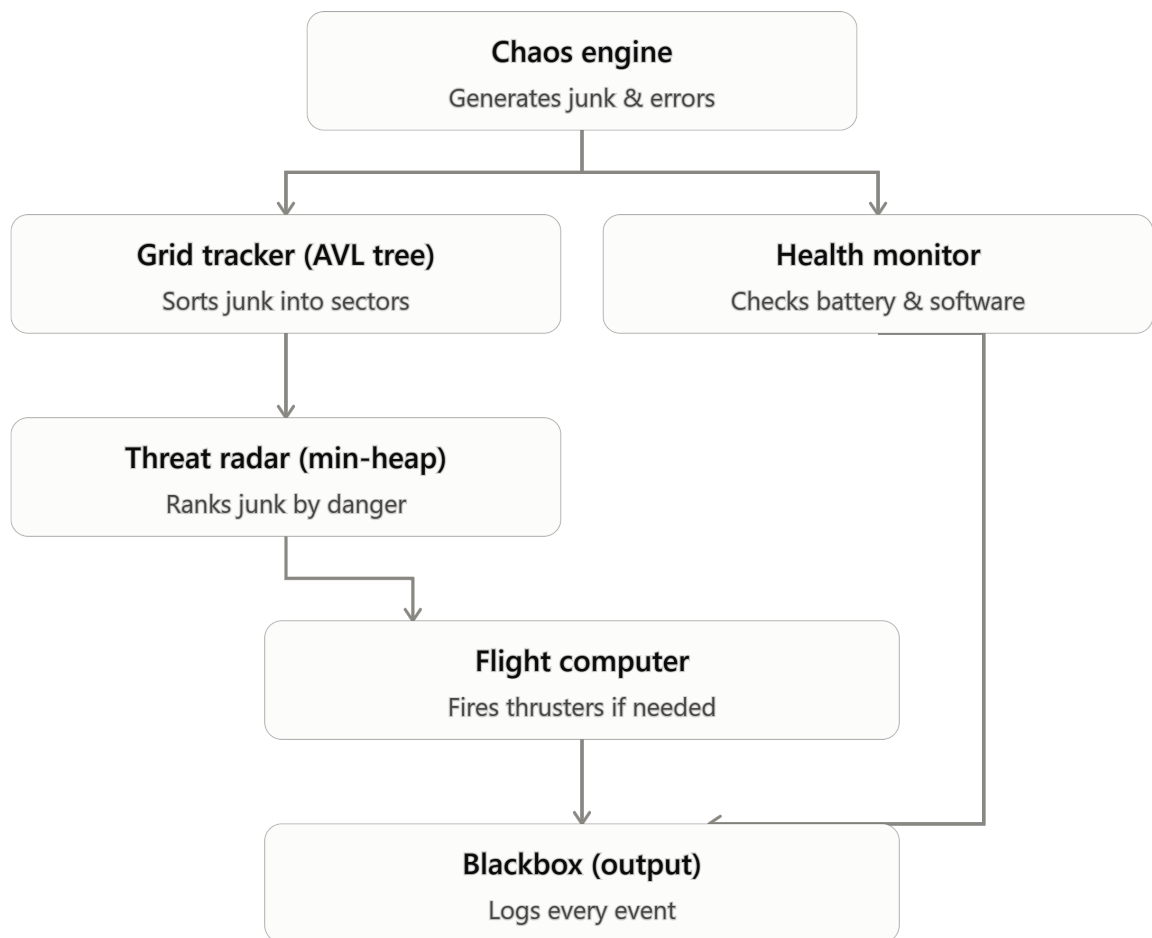


Fig. Project Architecture

Tasks Completed

Task Completed	Team Member
- Set up <code>Satellites_OOP.h</code> with base/derived classes. - Added fuel consumption logic to the satellite class. - Modified the status line to accept and display nearest friendly satellite data.	Chhavit
- Developed <code>Chaos_Engine.h</code> for random events. - Implemented <code>FlightComputer.h</code> logic. - Implemented the post-simulation satellite searching function by name.	Harsh Jain
- Developed <code>AVL_Tree.h</code> for 3D spatial grid mapping. - Wrote the <code>Vector3</code> math logic. - Added simulated gravity math to create circular orbits instead of straight lines.	Satyam Kumar
- Developed <code>Queues_DSA.h</code> (Min-Heap & Queues). - Added distance calculations between nearest friendly satellites. - Improved user input checks and buffer clearing.	Sudhanshu Ojha

Challenges/Roadblocks

1. Git Merge Conflicts: At the start of Phase II, working in a single C++ file caused major merge conflicts on GitHub. We solved this by separating the project into individual module files.

2. Input Buffer Crashes: The simulation used to crash endlessly if a user accidentally typed a letter (like "A") instead of a number during setup. We fixed this by implementing standard `cin.clear()` and `cin.ignore()` while loops to strictly validate inputs.

3.As for the physics of linear orbits:The satellites initially moved in straight lines; we therefore carried out some research and made the necessary changes to apply a simulated gravitational pull in the main loop to constantly bend their velocity vectors into a circular orbit.

Tasks Pending

Task Pending	Team Member (to complete the task)
Designing the new HTML/CSS based frontend web application that is in the phase III stage.	All Team Members
Reading the generated <code>telemetry.csv</code> file which has been generated by <code>orbital.cpp</code> for visual simulation.	All Team Members
Balancing the Chaos Engine probabilities so the web demonstration looks realistic.	Harsh & Satyam

Project Outcome/Deliverables

The project is currently a C++ program which operates from the command line and simulates satellites orbiting the Earth.

The main deliverables at this stage are:

1. The main file **orbital.cpp**, combines all the header modules together.
2. A **Flight Computer** that uses AVL Trees and Min-Heaps to identify nearby space debris and carries out evasive burns.
3. Making sure that there is a search facility enabling you to locate the satellite's status by name.
4. A **blackbox.txt** which stores all events and any alerts that occur in safe mode.
5. There is a file called **telemetry.csv** which records the X, Y, and Z coordinates at each tick in order to function as a data bridge for Phase III.
6. As a part of **Phase III** the deliverable is a frontend web application based on HTML/CSS to provide a visual demonstration of the simulation.

Progress Overview

The C++ backend of the project is 90 to 95% complete and functional. We successfully connected the AVL Tree, Min-Heap, and OOP-based satellite classes. The `FlightComputer` correctly handles collision math, and the `Chaos_Engine` generates random events perfectly.

With the recent additions of circular orbit physics, active fuel consumption, strict input validation, and the CSV telemetry exporter, the backend is 90-95% ready. Our primary focus now is to transition into Phase III. We are going to build an HTML/CSS frontend to read the CSV file and to make visual simulation .

Codebase Information

Repository Link: <https://github.com/Chhavit09/SHMCA>

Branch: main

Latest Commits:

- 204e3a8: Implemented satellite searching function by name (harshj1214-hj)
- d27d767: Added fuel consumption logic to satellite class (Chhavit09)
- 20bccdb: Added distance calculations between nearest satellites & Improved input checks (sudhanshuOjha07)
- df655b9: Modify getStatusLine to accept nearest satellite data (Chhavit09)
- fb739c6: Implemented collision threat handling in the simulation (Chhavit09)
- a1e1004: Implemented threat detection using MinHeap (sudhanshuOjha07)
- 58db73a: Implemented FlightComputer and AVLTree in simulation (harshj1214-hj)
- d170326: Added main file with Vector3 struct(maths logic) and distance calculation (kr-Satyam09)
- 9d1fff0: Added evasive burn action (kr-Satyam09)
- a7d91c5: Added MinHeap class (sudhanshuOjha07)
- daa8ead: Implement derived classes for communication and imaging satellites (Chhavit09)
- cb205e6: Implemented flight computer class containing collision risk classification function (harshj1214-hj)

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
AVL Tree Spatial Indexing	Pass	Debris is grouped correctly by 3D sectors.
Min-Heap Priority Sorting	Pass	The closest debris is successfully pushed to the top of the heap.
Flight Computer Evasion	Pass	Calculates risk levels correctly and fires thrusters.
Input Validation & Search	Pass	Safely rejects string inputs and locates satellites by exact name.
Telemetry File Export	Pass	The telemetry.csv correctly logs X, Y, Z data for Phase III.

Deliverables Progress

- Modular C++ Backend (OOP & DSA combination): **Completed**
- Flight Computer Decision Engine: **Completed**
- Chaos Engine Event Generator: **Completed**
- Blackbox & Telemetry File I/O (.txt / .csv): **Completed**
- HTML/CSS Visual Frontend Webapp: **Planned for Phase III**